

LeetCode 3090: Maximum Length Substring With Two Occurrences

Concept & Approach

Core Strategy: Sliding Window / Two Pointers technique using a frequency map.

To solve this problem efficiently, we maintain a variable-size sliding window defined by two pointers, l (left) and r (right).

- **State Maintenance:** We use a `vector<int> freq(26, 0)` to keep track of the occurrences of each lowercase English letter within the current window.
- **Expansion:** We iterate the r pointer forward, adding the character `s[r]` to our window by incrementing its frequency.
- **Contraction:** If adding a character causes its frequency to exceed 2, the window becomes invalid. We then increment the l pointer, decrementing the frequency of `s[l]` until the window is valid again.
- **Result Update:** At each valid step, the size of the window is calculated as $r - l + 1$. We update our maximum length (ans) accordingly.

C++ Solution Code

```
class Solution {
public:
    int maximumLengthSubstring(string s) {

        int n = s.size();
        vector<int> freq(26, 0);

        int l = 0;
        int r = 0;

        int ans = 0;
        while(r < n) {

            freq[s[r] - 'a']++;
            while((freq[s[r] - 'a']) > 2) {
                freq[s[l] - 'a']--;
                l++;
            }

            ans = max(ans, r - l + 1);
            r++;
        }

        return ans;
    }
};
```

Note: Code indentation and spacing have been cleanly formatted for readability without altering logic.